Research Notes on Neural style transfer

Neural style transfer

>> Section 1

L content (p → , x → , l) = 1 2 ∑ i , j (A i j l (x →) − A i j l (p →)) 2 {\displaystyle {\mathcal {L}}_{\text{content }}({\vec {p}},{\vec {x}},l)={\frac {1}{2}}\sum _{i,j}\left(A_{ij}^{l}({\vec {x}})-A_{ij}^{l}({\vec {p}})\right)^{2}}

>> Section 2

Neural style transfer (NST) software algorithms are able to manipulate digital images, or videos, in order to adopt the appearance or visual style of another image. NST algorithms are characterized by their use of deep neural networks for the sake of image transformation. Common uses for NST are the creation of artificial artwork from photographs, for example by transferring the appearance of famous paintings to user-supplied photographs. Several notable mobile apps use NST techniques for this purpose, including DeepArt and Prisma. This method has been used by artists and designers around the globe to develop new artwork based on existent style(s).

>> Section 3

Symbols Let x → {\textstyle {\vec {x}}} be an image input to a CNN. Let F l ∈ R N l × M l {\textstyle F^{l}\in \mathbb {R} ^{N_{l}\times M_{l}}} be the matrix of filter responses in layer l {\textstyle l} to the image x → {\textstyle {\vec {x}}} , where:

>> Section 4

Extensions In some practical implementations, it is noted that the resulting image has too much high-frequency artifact, which can be suppressed by adding the total variation to the total loss. Compared to VGGNet, AlexNet does not work well for neural style transfer. NST has also been extended to videos. Subsequent work improved the speed of NST for images by using special-purpose normalizations. In a paper by Fei-Fei Li et al. adopted a different regularized loss metric and accelerated method for training to produce results in real-time (three orders of magnitude faster than Gatys). Their idea was to use not the pixel-based loss defined above but rather a 'perceptual loss' measuring the differences between higher-level layers within the CNN. They used a symmetric convolution-deconvolution CNN. Training uses a similar loss function to the basic NST method but also regularizes the output for smoothness using a total variation (TV) loss. Once trained, the network may be used to transform an image into the style used during training, using a single feed-forward pass of the network. However the network is restricted to the single style in which it has been trained. In a work by Chen Dongdong et al. they explored the fusion of optical flow information into feedforward networks in order to improve the temporal coherence of the output. Most recently, feature transform based NST methods have been explored for fast stylization that are not coupled to single specific style and enable user-controllable blending of styles, for example the whitening and coloring transform (WCT).

>> Section 5

F i j l (x →) {\textstyle F_{ij}^{l}({\vec {x}})} is the activation of the i th {\textstyle i^{\text{th}}} filter at position j {\textstyle j} in layer l {\textstyle l} . A given input image x → {\textstyle {\vec {x}}} is encoded in each layer of the CNN by the filter responses to that image, with higher layers encoding more global features, but losing details on local features.

>> Section 6

Content loss Let p → {\textstyle {\vec {p}}} be an original image. Let x → {\textstyle {\vec {x}}} be an image that is generated to match the content of p → {\textstyle {\vec {p}}} . Let P l {\textstyle P^{l}} be the matrix of filter responses in layer l {\textstyle l} to the image p → {\textstyle {\vec {p}}} . The content loss is defined as the squared-error loss between the feature representations of the generated image and the content image at a chosen layer l {\displaystyle l} of a CNN:

>> Section 7

Training Image x → {\displaystyle {\vec {x}}} is initially approximated by adding a small amount of white noise to input image p → {\displaystyle {\vec {p}}} and feeding it through the CNN. Then we successively backpropagate this loss through the network with the CNN weights fixed in order to update the pixels of x → {\displaystyle {\vec {x}}} . After several thousand epochs of training, an x → {\displaystyle {\vec {x}}} (hopefully) emerges that matches the style of a → {\displaystyle {\vec {a}}} and the content of p → {\displaystyle {\vec {p}}} . As of 2017, when implemented on a GPU, it takes a few minutes to converge.

>> Section 8

Minimizing this loss encourages the generated image to have similar style characteristics to the style image, as captured by the correlations between feature responses in each layer. The idea is that activation pattern correlations between filters in a single layer captures the "style" on the order of the receptive fields at that layer. Similarly to the previous case, the w l {\displaystyle w_{l}} are positive real numbers chosen as hyperparameters.

>> Section 9

Overview The idea of Neural Style Transfer (NST) is to take two images—a content image p → {\displaystyle {\vec {p}}} and a style image a → {\displaystyle {\vec {a}}} —and generate a third image x → {\displaystyle {\vec {x}}} that minimizes a weighted combination of two loss functions: a content loss L content (p → , x →) {\displaystyle {\mathcal {L}}_{\text{content }}({\vec {p}},{\vec {x}})} and a style loss L style (a → , x →) {\displaystyle {\mathcal {L}}_{\text{style }}({\vec {a}},{\vec {x}})} . The total loss is a linear sum of the two: L NST (p → , a → , x →) = α L content (p → , x →) + β L style (a → , x →) {\displaystyle {\mathcal {L}}_{\text{NST}}({\vec {p}},{\vec {a}},{\vec {x}})=\alpha {\mathcal {L}}_{\text{content}}({\vec {p}},{\vec {x}})+\beta {\mathcal {L}}_{\text{style}}({\vec {a}},{\vec {x}})} By jointly minimizing the content and style losses, NST generates an image that blends the content of the content image with the style of the style image. Both the content loss and the style loss measures the similarity of two images. The content similarity is the weighted sum of squared-differences between the neural activations of a single convolutional neural network (CNN) on two images. The style similarity is the weighted sum of Gram matrices within each layer (see below for details). The original paper used a VGG-19 CNN, but the method works for any CNN.